

Moodify: Emotion-Aware Music Recommendation Using Transformer-Based Language Models

Bengisu Duru Göksu

Department of Computer Engineering
Gebze Technical University
Gebze, Kocaeli, Turkey
b.goksu2022@gtu.edu.tr

Zeynep Yiğit

Department of Computer Engineering
Gebze Technical University
Gebze, Kocaeli, Turkey
z.yigit2022@gtu.edu.tr

Selin Bardakcı

Department of Computer Engineering
Gebze Technical University
Gebze, Kocaeli, Turkey
s.bardakci2022@gtu.edu.tr

Abstract—Emotion-aware recommendation systems have gained increasing attention with the rise of personalized digital experiences. In this paper, we present Moodify, a mobile application that detects user emotions from text or speech input and recommends music playlists accordingly. The system employs a transformer-based multi-label emotion classification model fine-tuned on the GoEmotions dataset to capture nuanced emotional states. Detected emotions are used to retrieve contextually relevant playlists via the Spotify Web API, while a lightweight large language model generates short, empathetic natural language responses to enhance user interaction. Experimental evaluations demonstrate that the proposed approach produces stable and meaningful emotion predictions, enabling effective emotion-driven music recommendation in real-world mobile scenarios.

Index Terms—emotion recognition, music recommendation, transformer models, mobile applications, natural language processing

I. INTRODUCTION

Music plays a significant role in the regulation of human emotions and the support of mental well-being. With the growing popularity of mobile applications and streaming platforms, personalized music recommendation systems have become an important research topic. Traditional recommendation approaches mainly rely on listening history or collaborative filtering, often neglecting the user’s current emotional state.

Recent advances in natural language processing have enabled more accurate emotion detection from textual and speech-based input. Transformer-based language models, in particular, have demonstrated strong performance in capturing contextual and subtle emotional cues. Leveraging these developments, this study introduces Moodify, an emotion-aware music recommendation system designed for mobile environments.

Moodify analyzes user-provided text or speech to identify multiple emotional states simultaneously and recommends music playlists that align with the detected emotions. The system integrates a fine-tuned transformer-based emotion classifier, Spotify’s music catalog, and a lightweight language model to generate empathetic responses. By combining emotion recognition with real-time music recommendation, Moodify aims to provide a more engaging and emotionally adaptive user experience.

II. ABBREVIATIONS AND ACRONYMS

The abbreviations and acronyms used throughout this paper are defined in their first appearance. ASR refers to Automatic Speech Recognition, which is used to convert voice input into text. NLP denotes Natural Language Processing, a field that enables emotion analysis from textual data. LLM stands for Large Language Model and is used to generate short, empathetic natural language responses. API refers to the Application Programming Interface, which enables communication between system components and external services such as Spotify.

III. EASE OF USE

Moodify is designed with a strong focus on usability and seamless user interaction, particularly for mobile environments. The application aims to minimize user effort by allowing emotion detection through short text input or voice recordings, without requiring explicit emotion selection. This design choice enables for a more natural and intuitive interaction flow.

From a system perspective, the architecture separates user-facing components from backend services to ensure responsiveness and scalability. Emotion analysis, music retrieval, and response generation are automatically handled in the background, allowing users to receive personalized music recommendations with minimal delay. By abstracting technical complexity away from the user, Moodify provides an accessible and emotionally adaptive music recommendation experience.

IV. SYSTEM OVERVIEW

Moodify follows a modular client-server architecture tailored for real-time emotion-aware music recommendation. The system is composed of three main components: a mobile client interface, a backend service layer, and an emotion classification service. This separation allows each component to operate independently while maintaining efficient communication throughout the system.

User input is provided as text or voice recordings through the mobile interface. Voice inputs are first transcribed into text using an automatic speech recognition service. The resulting text is then forwarded to the emotion classification

service, which predicts multiple emotional states simultaneously. Based on the detected primary emotion, the backend retrieves relevant music playlists from the Spotify Web API and generates a short, empathetic response to accompany the recommendation.

This design enables scalability, fault tolerance, and low-latency interaction, making Moodify suitable for mobile environments where responsiveness and usability are critical.

V. SYSTEM ARCHITECTURE

The Moodify system is designed following a modular client-server architecture that enables real-time, emotion-aware music recommendation while maintaining scalability and low latency. The architecture separates user interaction, service orchestration, emotion analysis, and recommendation generation into independent components, each responsible for a well-defined task. This design allows flexible integration of external services, such as speech recognition and music streaming APIs, while ensuring that computationally intensive operations are handled outside the mobile client.

A. Mobile Client (User Input Layer)

The mobile client serves as the primary user interaction layer of the Moodify system. It is responsible for collecting user input, presenting recommended music playlists, and displaying emotionally adaptive textual responses generated by the backend services. The client is intentionally designed to remain lightweight, delegating all computationally intensive tasks to backend components.

Users can provide emotional input through two modalities: textual input and voice recordings. In the text-based interaction mode, short user messages that describe their current feelings are directly transmitted to the backend API as JSON payloads. In voice-based interaction mode, the mobile client records short audio clips and uploads them to the backend using a multipart HTTP request. The client does not perform any on-device speech processing or emotion inference, thereby reducing resource consumption and ensuring consistent behavior across devices.

After the backend completes emotion analysis, playlist retrieval, and response generation, the mobile client receives a structured JSON response containing the detected primary emotion, auxiliary emotional signals, recommended playlists, and a short empathetic message. This response is rendered directly in the user interface without additional processing. By limiting the client's responsibilities to input collection and result presentation, the system achieves a clear separation of concerns and maintains responsiveness in mobile environments.

B. Backend API (Orchestration Layer)

The backend API constitutes the central orchestration layer of the Moodify system. Implemented using Node.js and the Express framework, this component coordinates communication between the mobile client, the emotion recognition service, external APIs, and the response generation module.

The backend is designed as a stateless RESTful service, enabling scalability and simplifying deployment in distributed environments.

Upon receiving user input from the mobile client, the backend determines the appropriate processing path based on the input modality. Text-based inputs are forwarded directly to the emotion model service for inference, while voice-based inputs are first routed to the speech-to-text service for transcription. The backend then aggregates the outputs of downstream services and manages the overall control flow of the system.

The orchestration logic includes explicit error handling and fallback mechanisms to ensure robust operation under partial service failures. For instance, if the emotion model service is unreachable or returns an invalid response, the backend gracefully degrades by assigning a neutral emotional state. This design choice prevents user-facing failures and maintains a consistent interaction experience.

In addition to emotion analysis coordination, the backend is responsible for integrating with the Spotify Web API to retrieve emotion-aligned playlists. Authentication tokens are managed server-side, and search queries are dynamically constructed based on the detected primary emotion. Finally, the backend invokes a lightweight large language model to generate a short, empathetic natural language response that complements the recommended playlist.

By encapsulating service coordination, error handling, and external API integration within a single orchestration layer, the backend API enables a clean separation of concerns and keeps the mobile client independent of system-level complexity.

C. Speech-to-Text Service (Automatic Speech Recognition)

The speech-to-text service enables voice-based interaction by converting short user audio recordings into textual input suitable for downstream emotion analysis. This component is implemented as a backend service and utilizes a cloud-based automatic speech recognition model to ensure high transcription accuracy without imposing computational overhead on the mobile client.

Audio recordings are transmitted from the mobile client to the backend API as multipart HTTP requests. Upon receipt, the backend temporarily stores the uploaded audio file and forwards it to the speech recognition service for transcription. The system supports multiple common audio formats and enforces strict file size limits to ensure reliability and predictable processing latency.

The transcription process is performed using a pretrained speech recognition model that returns a plain-text representation of the spoken content. Language parameters can be specified to improve transcription accuracy. Once transcription is completed, the temporary audio file is immediately deleted to minimize storage usage and reduce privacy risks.

If the transcription process fails or produces empty output, the service returns an explicit error response, preventing invalid data from propagating to subsequent components. Successfully transcribed text is returned to the backend API and

treated identically to direct textual input provided by the user. By isolating speech recognition as an independent service, the system maintains modularity and allows future replacement or extension of the ASR component without affecting the rest of the architecture.

D. Emotion Model Service (FastAPI)

The Emotion Model Service constitutes the core intelligence component of the Moodify system and is responsible for predicting users' emotional states from textual input. This service is deployed as an independent FastAPI microservice and is accessed by the backend orchestration layer through a dedicated `/predict` endpoint. Decoupling emotion inference from the main backend enables independent scaling, simplified deployment, and flexible model iteration without impacting other system components.

Emotion detection is formulated as a multi-label text classification task, as a single user utterance may simultaneously express multiple emotional states. The model is fine-tuned on the GoEmotions dataset, which contains short user-generated texts annotated with 28 distinct emotion categories. Each emotion is modeled as an independent binary prediction, and the output layer applies a sigmoid activation function to produce per-class confidence scores.

The training objective is optimized using Binary Cross-Entropy (BCE) loss, which is well-suited for multi-label classification and allows each emotion category to be learned independently. During preprocessing, input text is tokenized with truncation and fixed-length padding up to 128 tokens to ensure consistent model input. Model performance is primarily monitored using the Micro F1-score, as it provides a balanced evaluation of precision and recall across all emotion labels, including infrequent classes.

While multiple transformer-based architectures were explored during development, the system employs a fine-tuned BERT-base model for production inference. BERT-base preserves the full bidirectional attention mechanism and deeper transformer stack, enabling richer contextual representations of nuanced emotional expressions. This capability is particularly important for detecting compound emotions and handling linguistic phenomena such as negation, tone shifts, and subtle affective cues.

The trained model is loaded once at service startup and kept in evaluation mode to ensure efficient inference. Incoming text is tokenized on request, passed through the model to obtain raw logits, and converted into probability scores via sigmoid activation. The service returns a structured prediction containing a primary emotion label and a ranked list of secondary emotions with associated confidence values.

E. Inference Optimization Layer

The Inference Optimization Layer is implemented as a dedicated post-processing component that operates on the raw probability outputs of the transformer-based emotion classifier. Since the underlying model produces independent confidence scores for each emotion label via sigmoid activation, directly

applying a fixed global threshold may result in unstable or noisy predictions, particularly in multi-label and real-time settings. This layer addresses such challenges through a series of deterministic inference-time calibration strategies.

First, class-specific decision thresholds are applied instead of a single global cutoff. These thresholds are empirically optimized on validation data and stored in an external configuration file, enabling each emotion category to be calibrated independently according to its prediction behavior. This approach significantly reduces false positives for frequently over-predicted emotions while preserving recall for more subtle classes.

Second, a minimum confidence constraint is enforced to filter out low-probability predictions that are unlikely to represent meaningful emotional signals. In addition, the system applies a top- K selection strategy (with $K = 3$), retaining only the highest-confidence emotion predictions. This restriction ensures that downstream components, such as playlist retrieval and response generation, operate on a compact and interpretable emotional representation rather than an excessively broad label set.

To account for limited contextual information in very short user inputs, the optimization layer introduces a length-aware threshold adjustment. For utterances below a predefined word count, a small penalty term is added to the decision thresholds, making the classifier more conservative when insufficient linguistic context is available. This mechanism improves robustness in mobile interaction scenarios where users often provide brief or fragmented inputs.

Finally, the optimized binary predictions are used to compute standard evaluation metrics, including Micro, Macro, and Weighted F1-scores, as well as accuracy, precision, and recall, during inference-time evaluation. By separating inference optimization from both model training and core classification, this layer enhances prediction stability and interpretability without increasing model complexity or retraining costs.

F. Spotify Web API (Music Recommendation Service)

The music recommendation component of Moodify integrates the Spotify Web API to retrieve playlists aligned with the user's detected emotional state. This service is accessed exclusively through the backend API, ensuring that authentication credentials and access tokens are never exposed to the mobile client.

Authentication with Spotify is performed using the OAuth 2.0 Client Credentials flow. Upon the first request, the backend requests an access token from the Spotify authorization server by encoding the client identifier and secret as a base64 authorization header. The received token is cached in memory together with its expiration timestamp, allowing subsequent requests to reuse the same token until it expires. This caching mechanism significantly reduces authentication overhead, minimizes external API calls, and improves overall response latency.

When a valid emotion prediction is available, the backend constructs a search query using the detected primary emotion

as a keyword and requests a limited number of public playlists from Spotify’s search endpoint. Retrieved playlist metadata is filtered and normalized before being passed to downstream components, ensuring that only relevant fields such as playlist name, external URL, and cover image are retained.

By centralizing playlist retrieval and token management within the backend, the system achieves secure credential handling, reduced network overhead, and consistent recommendation behavior. This design also allows the recommendation logic to remain loosely coupled from the emotion detection pipeline, enabling future extensions such as personalized ranking or multi-emotion blending without changes to the client application.

G. LLM Response Generator (OpenAI GPT-4.1-mini)

The LLM Response Generator is responsible for producing short, emotionally adaptive natural language messages that accompany the recommended music playlists. Rather than performing reasoning or decision-making, this component functions as a surface-level language generator that transforms structured emotional and recommendation data into human-like textual responses.

The service is implemented within the backend API and utilizes a lightweight large language model, GPT-4.1-mini, selected to balance linguistic quality with low latency and predictable response times. Deploying the LLM on the server side avoids exposing API credentials to the mobile client and enables centralized prompt control and response filtering.

Input to the language model consists of the detected primary emotion, a set of secondary emotions, and metadata for a single selected playlist. These elements are embedded into a carefully designed prompt that constrains the model’s output style and length. System-level instructions enforce a non-generic, empathetic tone while explicitly limiting the response to two or three sentences and restricting references to a single playlist. This controlled prompting strategy ensures consistency across responses and prevents verbose or promotional output that could degrade user experience.

Model generation parameters are configured to further stabilize output behavior. A maximum token limit bounds response length, while a moderate temperature value encourages natural phrasing without introducing excessive variability. The generated message is returned as part of a structured JSON response alongside emotion predictions and playlist metadata.

By restricting the role of the LLM to natural language realization, the system preserves a clear separation between decision logic and language generation. This design allows Moodify to leverage the expressive capabilities of large language models while maintaining transparency, controllability, and real-time performance in a mobile setting.

H. Structured JSON Response

The final output of the Moodify system is returned to the mobile client as a structured JSON response that encapsulates the results of emotion analysis, music recommendation, and

natural language generation. This standardized response format establishes a clear and stable contract between the backend services and the client application, enabling simple rendering logic on the client side without additional processing.

The response includes four main fields: the detected primary emotion, a ranked list of predicted emotions with associated confidence scores, a list of recommended playlists retrieved from the Spotify Web API, and a short natural language message generated by the LLM Response Generator. Playlist objects are represented using minimal metadata, including the playlist name, external Spotify URL, and optional cover image, ensuring efficient data transfer and fast rendering in mobile environments.

By aggregating outputs from multiple backend components into a single JSON object, the system avoids multiple round-trip requests and reduces network overhead. This design also allows the mobile client to remain agnostic to internal system complexity, focusing solely on user interaction and presentation. The structured response format further supports extensibility, allowing additional fields to be introduced in future iterations without breaking compatibility with existing clients.

VI. EMOTION MODEL EVALUATION AND SELECTION

This section presents the experimental evaluation and model selection process conducted to identify the most suitable emotion classification model for the Moodify system. Given the central role of emotion inference in driving music recommendation quality, particular emphasis is placed on robustness, stability, and multi-label prediction performance rather than purely computational efficiency.

Emotion detection is formulated as a multi-label classification problem, as user utterances may simultaneously express multiple emotional states. All candidate models are fine-tuned on the GoEmotions dataset, which contains short user-generated texts annotated with 28 emotion categories. Each emotion is treated as an independent binary label, and model outputs are interpreted as per-class confidence scores.

Emotion detection is formulated as a multi-label classification problem, as user utterances may simultaneously express multiple emotional states. All candidate models are fine-tuned on the GoEmotions dataset, which contains short user-generated texts annotated with 28 emotion categories. Each emotion is treated as an independent binary label, and model outputs are interpreted as per-class confidence scores.

Both DistilBERT and BERT-base models are trained using an identical fine-tuning pipeline to ensure a fair and controlled comparison. Input texts are tokenized with truncation and fixed-length padding to a maximum sequence length of 128 tokens. The output layer applies a sigmoid activation function to enable independent probability estimation for each emotion category.

The models are optimized using Binary Cross-Entropy (BCE) loss, which is the standard objective for multi-label classification tasks and allows each emotion to be learned

independently. Model performance is primarily evaluated using the Micro F1-score, as it provides a balanced assessment of precision and recall across all emotion labels, including infrequent classes. Additional metrics such as Macro F1, Weighted F1, accuracy, precision, and recall are also reported to provide a comprehensive performance profile.

To accommodate architectural differences, batch sizes are adjusted according to model capacity and memory requirements. DistilBERT is trained with a larger batch size due to its compressed architecture, while BERT-base employs a smaller batch size to fit within GPU memory constraints. Apart from batch size, all other training hyperparameters are kept consistent across models.

DistilBERT is initially selected as a baseline model due to its lightweight architecture and reduced computational cost. As a distilled version of BERT, DistilBERT retains a significant portion of BERT’s representational power while offering faster inference and lower memory usage. These characteristics make it an attractive candidate for mobile-oriented systems where latency and resource constraints are critical.

However, empirical results indicate that DistilBERT struggles to provide sufficiently reliable performance for nuanced multi-label emotion detection. Despite threshold optimization during inference, the model exhibits lower precision and recall, particularly for compound emotional expressions and less frequent emotion categories. These limitations reduce the stability of downstream recommendations and negatively impact overall system reliability.

BERT-base employs the full transformer architecture with a deeper stack of attention layers and a larger number of trainable parameters compared to DistilBERT. This increased representational capacity enables the model to capture richer contextual information, including long-range dependencies, tone shifts, and subtle affective cues that are critical for accurate emotion inference.

Emotion classification frequently depends on nuanced linguistic patterns such as negation, contrastive phrasing, and co-occurring emotional signals. BERT-base preserves the original bidirectional attention mechanism without compression, allowing it to model interactions between words more effectively across the entire input sequence. This capability is particularly beneficial in multi-label settings where multiple emotions may coexist within a single sentence, as reflected in the inference-time comparison shown in Figure 1.

Figure 1 summarizes the inference-time performance for both models. BERT-base achieves a substantially higher Micro F1-score (75.70%) compared to DistilBERT (61.51%), indicating improved balance between precision and recall across all emotion labels. Gains are also observed in Macro F1 (70.53% vs. 48.52%), demonstrating superior robustness for less frequent emotion categories.

In addition to F1-score improvements, BERT-base outperforms DistilBERT in accuracy (59.19% vs. 46.31%), precision (74.93% vs. 60.36%), and recall (76.47% vs. 62.70%). These consistent improvements highlight the importance of stronger

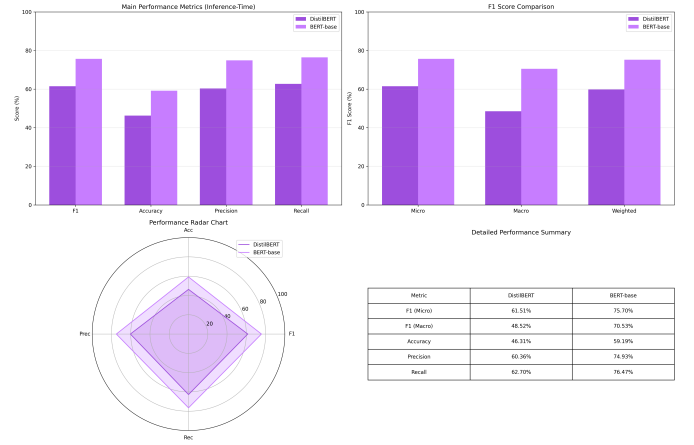


Fig. 1. Inference-time performance comparison between DistilBERT and BERT-base on the GoEmotions dataset. BERT-base consistently outperforms DistilBERT across all major evaluation metrics.

contextual representations for stable and interpretable emotion predictions in real-world usage scenarios.

Although DistilBERT offers computational efficiency and lower inference latency, the observed performance gap demonstrates that emotion-aware recommendation systems benefit more from expressive language representations than from lightweight architectures. In Moodify, inaccurate or unstable emotion predictions directly propagate to playlist selection and natural language response generation, amplifying their negative impact on user experience.

Based on empirical evaluation and system-level considerations, BERT-base is selected as the final emotion classification model for Moodify. The improved accuracy, robustness, and multi-label consistency of BERT-base outweigh its higher computational cost, particularly since inference is performed on the server side. This choice ensures reliable emotion detection and forms a stable foundation for emotion-driven music recommendation.

VII. SYSTEM PIPELINE

This section describes the end-to-end processing flow of the Moodify system, focusing on the runtime interaction between system components rather than their static structure. While the system architecture defines the roles and responsibilities of individual modules, the system pipeline illustrates how user input is sequentially transformed into emotion-aware music recommendations during execution.

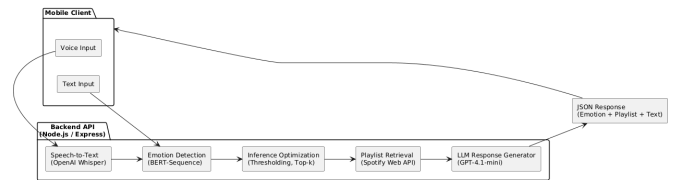


Fig. 2. End-to-end system pipeline of Moodify, illustrating the sequence of speech transcription, emotion analysis, playlist retrieval, and natural language response generation.

Fig. 2 presents the complete execution pipeline of Moodify. The pipeline begins at the mobile client and proceeds through a series of backend services, each responsible for a specific transformation of the user input. The final output consists of emotion-aligned playlist recommendations and a short empathetic message, which are returned to the client as a structured JSON response. The pipeline is initiated when the user provides input through the mobile application, either by entering a short text message or recording a voice message. Text-based inputs are transmitted directly to the backend API, whereas voice inputs are uploaded as audio files and processed by the speech-to-text service. Once voice input is transcribed into text, both input modalities follow an identical downstream processing path.

Input Modality Handling

The system pipeline is designed to support both text-based and speech-based interaction while maintaining a unified processing flow beyond the initial input stage. Voice input is converted into text at an early stage using an automatic speech recognition service, after which no distinction is made between spoken and written input. This early unification simplifies system orchestration, avoids duplication of emotion analysis logic, and ensures consistent emotion inference across different input modalities. By enforcing a single text-based representation for downstream processing, all subsequent components—including the emotion classification service, playlist retrieval logic, and response generation module—operate independently of the original input format. This design also facilitates future extensibility, allowing new input modalities to be integrated without requiring changes to the core pipeline.

Fault Tolerance and Degradation Strategy

To ensure robust operation in real-world usage scenarios, the pipeline incorporates explicit fault-tolerance mechanisms at critical stages. If the emotion classification service becomes temporarily unavailable or returns an invalid response, the backend assigns a neutral emotional state and continues execution without interrupting the overall pipeline. This graceful degradation strategy prevents partial service failures from propagating to the user interface and ensures uninterrupted delivery of music recommendations. Similar safeguards are applied during interactions with external services, such as playlist retrieval, enabling the system to maintain a stable and predictable behavior even under partial service outages.

Latency-Aware Design Considerations

The pipeline is structured to minimize perceived latency, which is particularly important for interactive mobile applications. Computationally intensive operations, including speech transcription, emotion inference, and natural language response generation, are executed exclusively on backend services rather than on the mobile client. This design allows the client to remain lightweight and responsive while leveraging more capable server-side resources. In addition, interactions with external APIs are optimized to reduce redundant network

overhead and improve response time. The backend manages authentication for third-party services centrally, caching access tokens where possible to avoid repeated authorization requests. For example, Spotify Web API access tokens are reused until expiration, eliminating unnecessary token exchanges for consecutive playlist queries. API requests are constructed using concise query parameters derived directly from the detected primary emotion, and only essential response fields are processed by the backend. By keeping external API calls stateless and minimizing payload size, the system reduces latency and improves reliability, which is particularly beneficial under variable mobile network conditions.

VIII. USER INTERFACE

This section presents the mobile user interface of Moodify and demonstrates how the outputs of the system pipeline are exposed to the user. The interface is designed to support intuitive interaction while remaining lightweight, with all computational logic executed on backend services.

The application consists of two primary screens: a landing interface for user input and a recommendation interface for displaying emotion-aware playlist suggestions and generated responses.

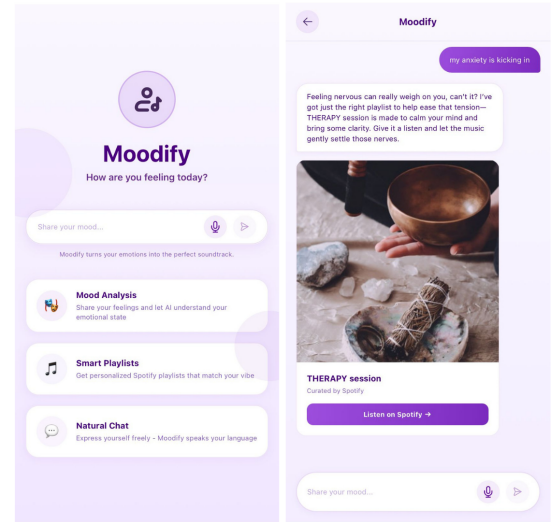


Fig. 3. Moodify mobile user interface. The top screen shows the landing page where users provide text or voice input. The bottom screen displays the recommended Spotify playlist and the empathetic response generated by the system.

Fig. 3 illustrates the complete user interaction flow. The landing page allows users to initiate the system pipeline by submitting textual or voice-based input. Once processing is completed, the recommendation screen presents an emotion-aligned playlist together with a short natural language message.

The user interface remains intentionally minimal, delegating all emotion inference, recommendation logic, and response generation to backend services. This separation ensures responsive interaction on mobile devices and preserves a clear boundary between presentation and system intelligence.

IX. LIMITATIONS

Despite its effectiveness in emotion-aware music recommendation, the current Moodify system has several limitations. First, the emotion classification pipeline is designed to always produce a prediction for any textual input, including emotionally neutral or ambiguous utterances such as greetings (e.g., “hello”). In such cases, the system may infer weak or spurious emotional signals, leading to recommendations that are not strongly grounded in the user’s actual emotional state.

Second, the system does not explicitly model user-specific preferences or historical listening behavior. All recommendations are generated based solely on the detected emotional context, without incorporating personalization factors such as past interactions or genre affinity. While this design simplifies the pipeline and preserves user privacy, it may limit recommendation relevance for long-term users.

Finally, emotion detection is currently limited to short, single-turn inputs and does not consider temporal emotional dynamics across multiple interactions. As a result, the system cannot distinguish between transient emotional expressions and more persistent affective states. Additionally, the emotion classifier is trained on English-language data, which restricts the system’s applicability in multilingual settings.

X. FUTURE WORK

Future work will primarily focus on improving the robustness and semantic reliability of the emotion inference pipeline. One important extension is the integration of an explicit emotion validity or intent detection stage prior to emotion classification. This component could be implemented as a lightweight binary classifier or rule-based filter that distinguishes emotionally neutral utterances (e.g., greetings or acknowledgments) from affective content. Inputs identified as non-emotional could bypass the emotion classifier or be mapped to a neutral state, preventing spurious emotion inference and improving recommendation relevance.

Another direction involves augmenting the emotion model with a calibrated confidence-based rejection mechanism. Instead of always producing emotion labels, the system could abstain from prediction when the maximum confidence score falls below a learned threshold. Such selective prediction strategies are well-suited for multi-label settings and would allow the pipeline to explicitly represent uncertainty in ambiguous or low-context inputs.

Future versions of Moodify may also incorporate user-level personalization to complement emotion-based recommendations. This could be achieved by introducing a lightweight preference modeling layer that aggregates anonymized interaction history, such as previously selected playlists or skipped recommendations. Combining short-term emotional context with longer-term preference signals would enable more nuanced recommendation strategies without requiring full collaborative filtering infrastructure.

In addition, extending the pipeline to support multi-turn emotion tracking represents a promising research direction. By maintaining a short temporal window of recent emotion

predictions, the system could distinguish transient emotional expressions from persistent affective states. This would allow smoother recommendation transitions and reduce sensitivity to isolated or noisy inputs.

Finally, future work includes expanding multilingual support by fine-tuning the emotion classification model on multilingual emotion datasets and exploring privacy-aware, on-device inference techniques. Lightweight transformer variants or model distillation approaches could enable partial local inference, reducing dependency on network connectivity while preserving acceptable prediction accuracy.

XI. CONCLUSION

In this paper, we presented Moodify, an emotion-aware music recommendation system designed for mobile environments. Moodify integrates transformer-based multi-label emotion detection with real-time music retrieval and natural language response generation to deliver emotionally adaptive recommendations based on short text or speech inputs. By leveraging a modular client–server architecture, the system cleanly separates user interaction, orchestration logic, emotion inference, and external API integration, enabling scalability, robustness, and low-latency interaction.

Experimental analysis demonstrated that BERT-base significantly outperforms lighter transformer variants for nuanced multi-label emotion prediction, justifying its selection despite higher computational cost. To further stabilize predictions in real-world usage, the system incorporates an inference optimization layer that calibrates confidence thresholds, filters low-probability outputs, and accounts for limited input context. These design choices improve interpretability and reliability without increasing model complexity or retraining overhead.

The end-to-end system pipeline highlights how diverse components—speech transcription, emotion analysis, playlist retrieval, and language generation—interact during runtime to transform raw user input into meaningful recommendations. By centralizing computation and API management on the backend, Moodify maintains a lightweight mobile client while ensuring secure credential handling and efficient external service interaction.

Overall, Moodify demonstrates the feasibility and effectiveness of combining emotion-aware natural language processing with music recommendation in a practical mobile application setting. The presented system design and implementation provide a solid foundation for future research on emotionally adaptive recommendation systems that balance model expressiveness, system efficiency, and user experience.

REFERENCES

- [1] Spotify AB, “Spotify web API documentation,” Spotify for Developers. [Online]. Available: <https://developer.spotify.com/documentation/web-api>. Accessed: 2025.
- [2] E. Demszky, D. Movshovitz-Attias, J. Ko, A. Cowen, G. Nemade, and S. Ravi, “GoEmotions: A dataset of fine-grained emotions,” in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2020.
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017.